FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

LayerIt!: Towards a Time-Aligned Framework for Composable Music Visualisation Layers

<Fernando Azeredo>

WORKING VERSION

Mestrado em Engenharia Informática e Computação

Supervisor: Prof. <António Sá Pinto>

August 30, 2026

Abstract

Music Information Retrieval (MIR) visualisation tools play a crucial role in music analysis, annotation, and evaluation, but existing solutions often make it difficult to combine audio, symbolic scores, and algorithmic outputs in a single, time-aligned view. This thesis presents LayerIt, a Python-based object-oriented (Object-Oriented (OO)) framework for generating custom score-aligned MIR visualisations by combining established libraries and tools, including librosa, matplotlib, Modusa, ScoreWarp, and BeatThis. LayerIt is built around a modular layer-composition model in which data sources, audio plots, score renderings, and algorithmic outputs are treated as independent components that can be assembled into tailored visualisations. The resulting figures are exported as Scalable Vector Graphics (SVG), enabling resolution-independent output, post-processing, and future integration with web-based or interactive applications. As a validation case, LayerIt is used to visualise beat tracking results alongside aligned score and audio representations, showing how the framework supports both beat annotation and beat tracking evaluation through discrete beat estimates and continuous confidence curves. More broadly, the project demonstrates a reusable foundation for future specialized MIR visualisation workflows.

Keywords: Music Information Retrieval • Visualisation • Audio-to-Score Alignment • Beat Tracking • SVG • Object-Oriented • Python

Acknowledgements

<Fernando Azeredo>

Contents

List of Figures

List of Tables

List of Acronyms

BA	Beat Annotation
BT	Beat Tracking
DAW	Digital Audio Workstation
GUI	Graphical User Interface
MEI	Music Encoding Initiative
MIDI	Musical Instrument Digital Interface
MIR	Music Information Retrieval
OO	Object-Oriented
CQT	Constant-Q Transform
SVG	Scalable Vector Graphics

Chapter 1

Introduction

1.1 Contextualization

MIR is a research field that develops computational methods to analyze, interpret, and retrieve musical information from performance recordings and symbolic representations. MIR bridges a wide range of disciplines, including musicology, signal processing, machine learning, and human-computer interaction. As a data-intensive field, visualisation plays a crucial role in MIR research, the how and what is visualised, shapes the conclusions we draw and guides further progress.

The challenge of visualisation in MIR is particularly significant because music is an inherently abstract and subjective medium. Translating music into the visual domain is a problem that spans millennia; throughout history, multiple systems of musical notation have been developed in attempts to capture sound in written form, yet this remains an evolving challenge. For MIR purposes, this means that musical data comes in a variety of forms: from audio recordings, to symbolic scores, to annotations of expressive qualities. Each of these require different visual approaches. An audio visualisation such as a waveform or spectrogram reveals how a performance sounds, while a musical score conveys how a piece should be performed; both serve distinct and important purposes for different research tasks.

1.2 Motivation

MIR research increasingly demands visualisations that integrate multiple modalities simultaneously. MIR researchers often need to inspect algorithmic outputs alongside audio representations and symbolic scores in a time-coherent manner. This kind of multi-modal, time-aligned visualisation is central to both the evaluation of MIR algorithms and the development of annotation workflows. Audio-to-score alignment, the task of synchronizing a performance recording with its corresponding symbolic score, is a key enabler of this type of visualisation, as it provides the temporal anchor needed to align heterogeneous data sources into a unified view.

This thesis was initiated by two closely related challenges within BT and BA research. Contemporary BT systems are predominantly deep learning-based thus, the field is deeply reliant on

tools to enable better execution of annotation and evaluation tasks, these are essential for advancing BT research.

Annotation tasks, and more concretely BA itself, remain a laborious and time-consuming process. Existing Graphical User Interface (GUI) tools such as Sonic Visualiser present a clear gap in this area: integrating newer state-of-the-art BT algorithms is difficult, and BA specific features have seen little meaningful innovation. This disconnect between annotation workflows and the algorithms they are meant to support is a concrete obstacle to progress in both domains.

For evaluation tasks, GUI tools suffer from a similar problem, where methods to visualise how developed algorithms perform on a deeper level are lacking. Providing better tools to evaluate and compare models between each other is also key to improving them.

Although solutions like Sonic Visualiser are powerful and widely used, their development cannot keep pace with the modern MIR research needs. Integrating new algorithms or producing specialized visualisations is often impractical within them. A particularly under served case is the visualisation of score-aligned data: combining audio and data representations with a musical score typically involves manual image manipulation, with no unified methodology in the community.

Because of these reasons, researchers frequently resort to *ad hoc* solutions that lack reproducibility and are difficult to share or extend, a fragmentation that represents a real obstacle to the collaboration which MIR research depends on. By contributing a flexible, extensible, and community-oriented visualisation tool built on established libraries, this thesis aims to not only address an immediate practical gap, but also to establish a foundation for a framework that can support the growing complexity and interdisciplinarity of MIR research.

All of the concerns described in this section serve as a basis for the objectives and research questions we explicitly present in the following section.

1.3 Objectives, Research Questions and Contributions

This thesis is guided by three objectives, which in turn lead to three research questions and a set of concrete contributions.

- **Objective 1**: Develop a reusable Python framework for composing score-aligned MIR visualisations based on an OO layer-composition model.
- **Objective 2**: Demonstrate that the framework can generate and regenerate score-aligned visualisation compositions from modular layers.
- **Objective 3**: Demonstrate how score-aligned visualisations can provide qualitative visual context for beat annotation and BT evaluation workflows.
- **RQ1**: How can heterogeneous MIR visualisations be combined into a common score-aligned representation?
- **RQ2**: How does LayerIt support the creation, modification, and regeneration of score-aligned visualisations through modular layers and scriptable composition?

- **RQ3**: How can score-aligned visualisations provide qualitative context for interpreting BT outputs and inspecting beat annotation data, and what alignment-related limitations affect this use?

The main contributions of this thesis are the following:

- The LayerIt framework for composing score-aligned MIR visualisations through an OO layer-composition model.
- The release of LayerIt as an open-source, pip-installable Python library.
- The open-source repository containing the five prototype visualisations developed as part of this work.
- A Late-Breaking Demo paper submitted to ISMIR 2026.

The contribution is therefore the layer-composition framework and the figures it enables, rather than the alignment method itself, which is provided by ScoreWarp.

1.4 Developed Project

To address the previously mentioned objectives and research questions, we developed LayerIt, a Python-based OO framework for composing score-aligned MIR visualisations. LayerIt integrates established MIR libraries and tools into an SVG-based workflow in which visual elements are treated as independent layers that can be composed, reordered, and combined into custom figures.

At its core, LayerIt supports the integration of symbolic scores with audio and algorithmic data in a single view. By relying on ScoreWarp for temporal alignment, it makes it possible to overlay spectrograms, onset information, and beat-tracking outputs on top of score representations within the same visualisation. The result is a flexible layer-composition model that supports the prototypes developed in this thesis and provides a foundation for future MIR visualisation workflows.

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** provides a literature review covering the main audio visualisation types, existing GUI and code-based visualisation tools, and audio-to-score alignment.
- **Chapter 3** presents the adopted methodology and describes the five iterative phases that culminated in the development of LayerIt.
- **Chapter 4** describes the problem addressed by LayerIt, its design and implementation, and the “layers within layers” philosophy that underpins its architecture and use.

- **Chapter 5** evaluates the framework against the research questions through the generated figures, a qualitative workflow comparison, and a discussion of its limitations.
- **Chapter 6** reflects on the achievements of the thesis and outlines directions for future work.

Chapter 2

Literature Review

In this chapter, we present the background required for this thesis. We begin by exploring the landscape of music visualisation methods, categorizing them into three main types: sheet music representations, symbolic machine-readable formats, and audio-based visualisations. This foundation is essential for understanding how different types of musical information can be represented visually and interacted with computationally. We then examine contemporary MIR visualisation tools, distinguishing between GUI-based solutions and code-based workflows to highlight the gap that motivates our work. Finally, we introduce audio-to-score alignment and explore relevant tools and approaches that have informed our solution.

2.1 Visualisation Types

We can sum up all music related visualisations into three categories:

1. Sheet Music and Score Images
2. Symbolic Representations
3. Audio Representations

2.1.1 Sheet Music and Score Images

Sheet music describes musical work using a formal language based on musical symbols and letters, thus, this kind of music representations always require a level of music literacy from the performing musician. Furthermore, sheet music usually does not induce explicit information on how a given piece should be performed, it is intended for the musician to have a certain level of interpretation liberty. This can include the variance of tempo, articulation and dynamics among others.

Sheet music also comes in a variety of forms that vary throughout History and cultures. For our purposes we are going to focus on the western standard of music notation. Still, within western music, there can be vastly different forms of notation, however, most are based on a staff which

sets five horizontal lines and four spaces, each representing a different musical pitch. Musical relevant symbols are put inside the staff to denote pitch, temporal and expressive indications. [**mullerFundamentalsMusicProcessing2015**]

2.1.2 Symbolic Representations

Symbolic Representations are machine-readable formats where musical events and expressions are explicitly encoded. These come in various formats, such as MIDI and MusicXML.

Visually we can identify two main applications for this category, these include: piano-roll representations and digital score representations.

Historically piano-roll representations come from so-called “player pianos”, these were self playing pianos that became quite popular in the late 19th to early 20th century. This was achieved by a mechanism that would hit keys of the piano through a continuous roll of paper with perforations. The roll would move over a reading system (known as a tracker bar) that would trigger the musical actions accordingly.

This concept of a continuous and explicit representation of music performance would later prove useful for digital applications. Such is the case for the Piano-Roll.

Today the **Piano-Roll** is a staple feature in any DAW as a visual and interactive tool to write and edit musical events in the form of Musical Instrument Digital Interface (MIDI) notes and attributes such as velocity and other expressive controls.

MusicXML refers to the XML based language to transcribe score music into the digital domain. There are multiple sub-variants of the MusicXML format for specific applications and there are other formats for digital scores, but MusicXML can be considered as a "universal" format accepted in most digital score editors.

MusicXML was developed to serve as a universal format for storing and sharing music scores across different music notation applications. Music notation digital editors then employ proprietary variants of this format for addressing the program’s specific characteristics. In some cases these may not even be xml based, but MusicXML format can be considered the universal format, generally accepted and exportable across virtually all of them. These include, among others:

- **.mscz and .mscx**: These are Musescore specific formats named for "MuseScore Format" and "Uncompressed MuseScore" formats respectively. Both xml based.
- **.dora** - "Dorico" specific format (non xml based)
- **.sib** - "Sibelius" proprietary format of company Avid (also non xml based)
- **.mei** - The MEI format, XML based.

The .mei format is the one that is essential for this thesis and the most useful for MIR purposes. MEI stands for both the XML-based score format in itself (.mei), and the community that develops it(Music Encoding Initiative). Citing from the MEI website’s introduction:

"The Music Encoding Initiative (MEI) is a community-driven, open-source effort to define a system for encoding musical documents in a machine-readable structure. MEI brings together specialists from various music research communities, (...) to define best practices for representing a broad range of musical documents and structures."[**MEI-Website**]

Important to note that MEI focuses on a "machine-readable structure". This structure relies on unique identifiers (@xml:id) that make every element in the score individually addressable, from individual noteheads to complex spanning elements like slurs and ties. However, more than a music score format, MEI also serves to keep various notation, performance and interpretation data not only stored but also linked to individual score elements. This is why the MEI format and it's community are so essential to MIR as a whole.

That being said, MEI is a purely data driven project, and thus in order for it to be visualised or interacted with it needs to be decoded into a "human-readable format". Thats where Verovio comes in. The MEI and Verovio projects are closely inter-linked for that reason. While MEI defines how musical information is stored, Verovio is the primary tool used to transform that data into interactive visualisations and performance-ready outputs.

Verovio is a comprehensive open-source library designed to serve as a native typesetting engine for the MEI format. Verovio's primary role is to engrave MEI data directly into SVG without losing rich metadata through intermediate format conversions. The usage of the SVG format is an essential one for multiple reasons, quoting from Verovio's reference book:

> "In a web environment, the addressability can be used for highlighting graphical elements such as notes or any other music symbols. One additional characteristic of SVG is that its XML tree can be structured as desired, and an innovative design feature of Verovio was to go further in the structuring of the output by leveraging this characteristic. Since Verovio implements the MEI structure internally, this key feature of SVG made it possible to preserve the MEI structure in the design of the SVG output in Verovio. Preserving the MEI structure in the SVG output is a considerable overhead in the rendering process but makes it a unique feature of Verovio.
>
> As a result, Verovio output in SVG is not the end of a unidirectional rendering process. Quite on the contrary, it should instead be seen as an intermediate layer standing between the MEI encoding and its rendering that can act as the cornerstone for a bidirectional interaction: from the encoding to the notation, but also from the notation to the encoding through the user interface." [**repertoireinternationaldessourcesmusicalesVerovioReferenceB**

To further demonstrate the usefulness of MEI and Verovio, we'll present next a quote from the article relative to Piano Precision: a GUI software for audio-to-score visualisation. We will later explore this tool further, for now the importance of this quote comes from demonstrating a practical application of MEI and Verovio.

> "Technically, what makes MEI especially useful is that it can be rendered to graphical SVG output while retaining its semantically meaningful XML hierarchy

and unique identifiers for every score element (e.g., ties, notes). Piano Precision renders the scores into SVG output using Verovio, an open-source toolkit for visualising MEI data. This makes it easy to locate interesting objects in a score and, e.g., highlight them in the rendering. The result is interactive images in which users can flip pages, zoom in and out of a score, and click on elements in a score to jump to their playback positions. ” [**jiangPIANOPRECISIONADVANCING2025**]

2.1.3 Audio Representations

Audio representations are graphs and/or plots that intend to represent sound. These may not be for strictly representing music, the intent is to translate sound from the audible to the visual domain. Audio is a term that refers to an acoustic sound that is turned into an electrical signal. Most audio representations, are thus digitally generated through applying mathematical equations to the audio signal, such as the Fourier Transform.

We can infer at least 3 components that define a given audio. These are, time, frequency and amplitude/energy. An audio visualisation will present explicit information of at least 2 of these.

Waveforms are the most common and one of the simplest ways to represent audio. It consists of a single line on a graph in a 2D plot where amplitude is the y-axis and time is in the x-axis. This representation does not provide frequency information, it only shows how amplitude of the audio changes through time.

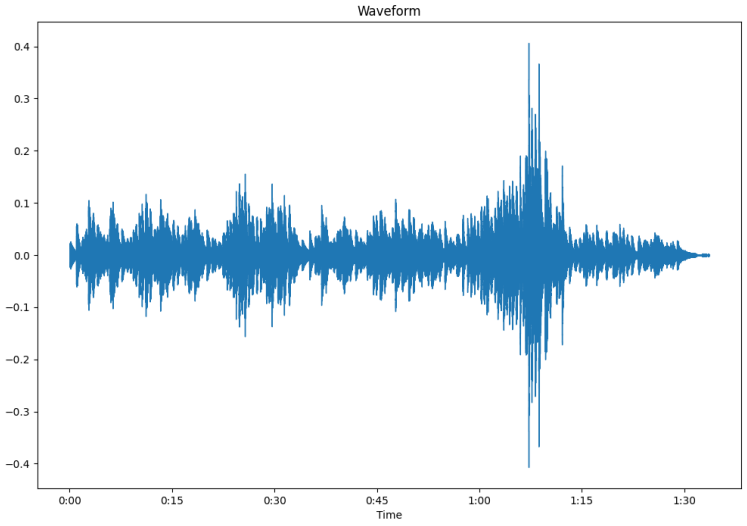

Figure 2.1: Example of an audio waveform.

A spectral visualisation, or a spectrum, refers to a graph with only frequency and amplitude/energy representation in a given time window.

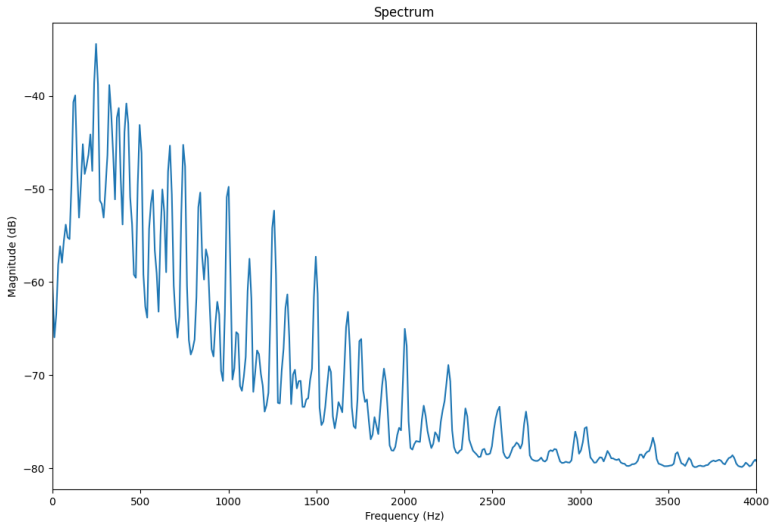

Figure 2.2: Example of audio Spectrum.

For displaying the three sound components the plot needs to represent three distinct dimensions. The spectrogram achieves this by having on the y axis the frequency, time on the x-axis and color as substitute for the third axis (amplitude), and in this way displaying all three parameters in a single 2D visual plot. There are multiple weights and variances of the spectrogram that have different applications.

The way frequencies are displayed in a given spectrogram can help understanding audio better for certain situations. There can be a wide variety of spectrogram representations, that can vary in how it's contents are displayed.

The linear spectrogram is the most basic form of spectrogram, where frequencies are displayed in a linear scale. Meaning, frequencies are equally spaced on the y-axis. For music applications, this kind of representation tends to be less useful, as it does not reflect the way humans perceive sound.

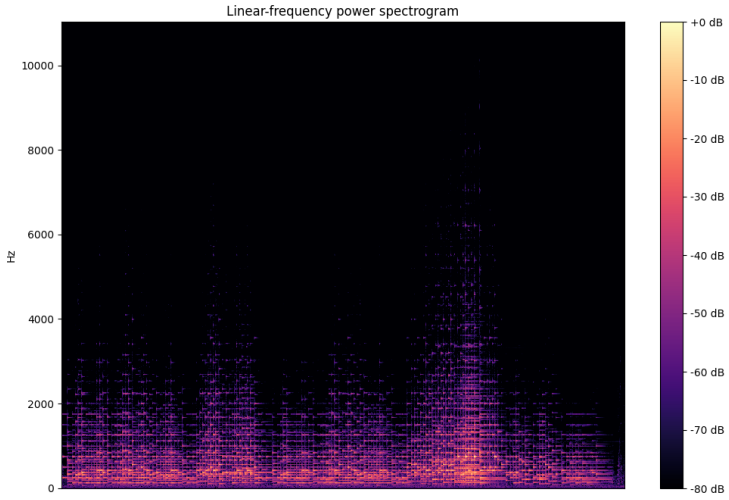

Figure 2.3: Example of Linear spectrogram.

The logarithmic spectrogram displays frequencies in a logarithmic scale. This mimics better the way humans perceive sound.

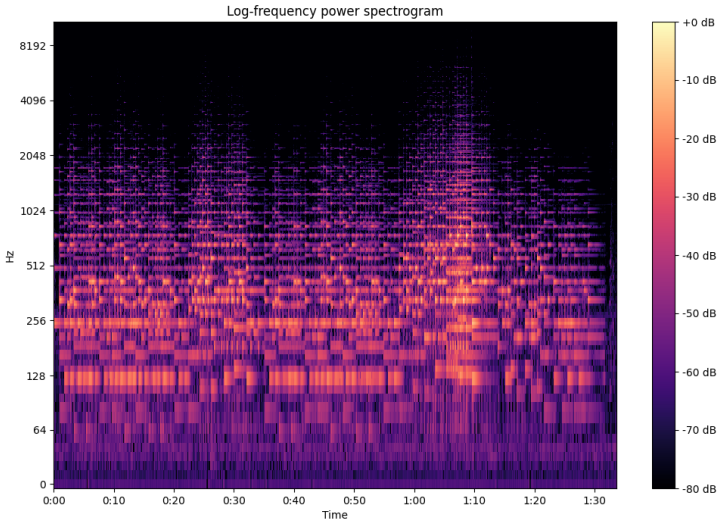

Figure 2.4: Example of Logarithmic spectrogram.

The Mel spectrogram also displays frequencies taking into account the human hearing curve. The difference being, that the logarithmic spectrogram simply organizes frequencies logarithmically. Whilst the Mel spectrogram organizes frequencies according to the Mel scale, which is a perceptual scale of pitches judged by listeners to be equal in distance from one another. For music related applications this type of spectrogram is much more useful, since we can more clearly distinguish in between musical pitches.

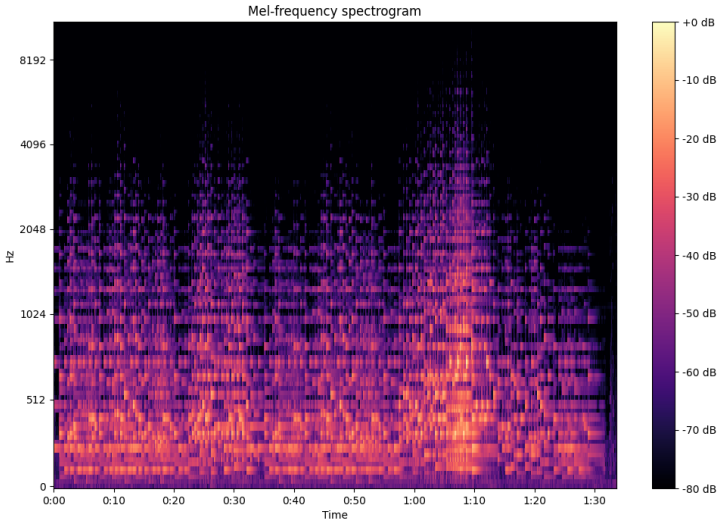

Figure 2.5: Example of Mel spectrogram.

The Constant-Q Transform (CQT) and chromagram are visual representations that derive from the spectrogram. Whereas a spectrogram shows frequency data, the CQT and chromagram can be used to visualise note-related information: tone height in the case of the CQT and chroma in the case of the chromagram. The following quote explains these terms:

> "(...) a pitch can be separated into two components, which are referred to as tone height and chroma. The tone height refers to the octave number and the chroma to the respective pitch spelling attribute contained in the set {C, C♯, D,(...), B}." [**mullerFundamentalsMusicProcessing2015**]

The CQT organizes its frequency content using geometrically spaced frequency bins. This is made in order to resemble the equal-tempered system. More commonly, the CQT can be interpreted as a Piano-Roll like representation.[**valleVisualDisplayRetrieval**]

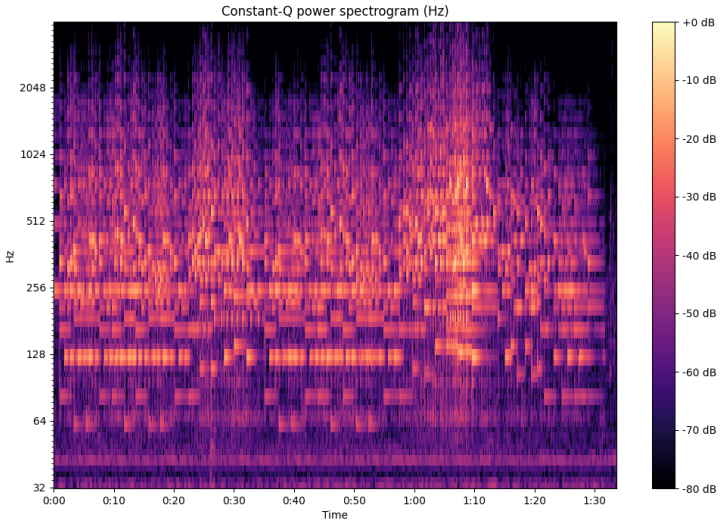

Figure 2.6: Example of a CQT.

While the CQT can distill the frequency spectrum into bins that correlate to individual notes, the chromagram takes this a step further by visually summing the energy of each note across all octaves. Thus in the end we get a plot divided into 12 lines, each representing a give note. In it's essence this type of visualisation serves to see the usage of the same notes across the recording. A chromagram can be useful for determining harmonic and scale related data.

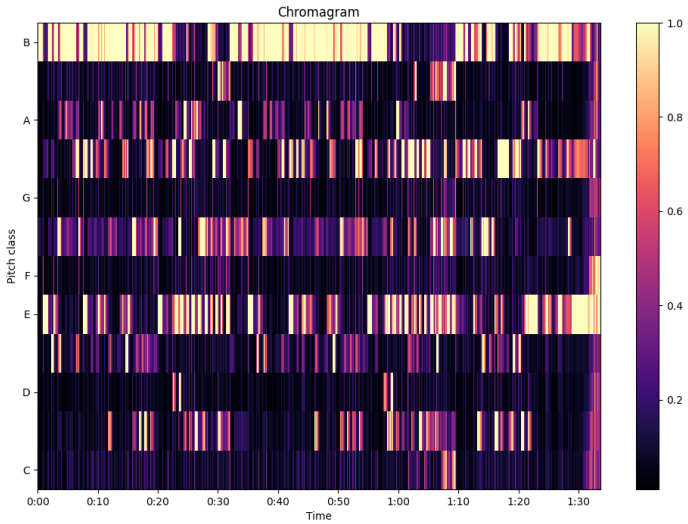

Figure 2.7: Example a Chromagram.

[mullerFundamentalsMusicProcessing2015, jungUnifiedCrossmodalTranslation2026a, khodzhaevPracti valleVisualDisplayRetrieval]

2.2 MIR Visualisation Tools

Within MIR, visualisation workflows typically fall into two broad categories: GUI-based tools and programming-based workflows. For this thesis, the most relevant comparison is between Sonic Visualiser as a representative GUI tool and code-based workflows used for custom MIR analysis.

Sonic Visualiser is a dedicated program for audio visualisation and analysis, and it is one of the most common tools in MIR practice. It is well suited to inspection, annotation, and exploratory analysis, making it a strong baseline for research-oriented visual work. A particularly relevant example is Piano Precision, a Sonic Visualiser based GUI tool for visualising score-aligned spectrograms.

> "The Piano Precision UI is adapted from Sonic Visualizer, with an additional area displaying a score (...) The interface is designed to help streamline the processes of loading a digital score and a performance recording, editing and visualising annotations (...), and exporting score-aware annotations."[**jiangPIANOPRECISIONADVANCING2025**]

While modern GUI-based tools such as Sonic Visualiser are powerful, they do not always provide the flexibility required for state-of-the-art MIR research. Many research questions demand highly specialized visualisations that are not available in pre-built software, especially when multiple data types such as audio, scores, annotations, and computational outputs must be combined in a single workflow. In these cases, programming-based methods are often necessary.

Code-based workflows are also attractive because they can be versioned, shared, and included directly in publications, which improves reproducibility. This is especially important in academic research, where results must be inspectable and repeatable. In addition, MIR increasingly overlaps with machine learning and large-scale data analysis, which naturally favours scripted workflows for feature extraction, training, evaluation, and custom visualisation.

Currently, Python is the go-to language for much of MIR research. Libraries such as Librosa, Matplotlib, and Madmom, together with tools such as Jupyter Notebooks, are commonly used to implement these workflows.

Taken together, these reasons explain why code-based visualisation remains essential in MIR and why Piano Precision is a useful motivating example for this thesis: it sits precisely at the intersection of interactive GUI support, audio-to-score alignment, and reproducible computational workflows.

2.3 Audio-to-score alignment

Audio-to-score alignment, also referred to as score matching or score alignment, consists of synchronizing an audio recording of a musical performance with its corresponding symbolic score [**carabias-ortiAUDIOSCOREALIGNMENTa**]. In practical terms, the goal is to establish correspondences between what is heard in the recording and what is represented in the score, enabling

tasks such as score following, guided listening, interactive navigation between audio and score, and research-oriented analysis.

TimeToAlign distinguishes graphical, logical, and physical temporal domains and provides a general model for recording alignments among them [**hentschelTimeAlignModelling2026**]. In this model, an alignment is a reusable artefact that records correspondences and their provenance across timelines. LayerIt addresses a complementary problem: it uses an available score-performance alignment to compose graphical, symbolic, and audio-derived representations into a shared score-aligned visualisation.

The purpose of an alignment is not limited to algorithmic matching. Thomas et al. [**thomasLinkingSheetMusic** describe score–audio linking structures as a basis for accessing and exploring multimodal music sources within a single framework. LayerIt uses an available linking structure for a more specific purpose: the generation of static, score-aligned visualisations. It does not compute the alignment itself. Existing synchronisation toolkits, such as Sync Toolbox, provide dynamic-time-warping-based pipelines that could supply an upstream alignment in future work [**mullerSyncToolbox2021**].

In the literature, it is useful to distinguish between two broad alignment strategies. The first is **time-warped alignment**, in which the score is mapped onto a continuous performance timeline. This approach preserves a direct temporal relationship between score elements and audio playback, and it is especially relevant when every relevant musical event needs to be placed along the time axis. The second is **link-based alignment**, in which the relationship between score and performance is established through discrete anchor correspondences. Rather than continuously warping the score, this approach links selected score positions to performance positions and uses those reference points to organize the alignment.

These two strategies are not separate tasks so much as different ways of representing the same underlying correspondence problem, and the distinction becomes important for the rest of this thesis. Time-warped alignment is the more continuous formulation, while link-based alignment is the more anchor-driven formulation. This terminology will be used throughout the following chapters.

Alignment systems can also be classified by when the matching is performed. **Offline alignment** processes a complete recording after the performance has finished. Because it can access the full signal, offline alignment can use future information to determine the best possible correspondence, which makes it suitable for analysis, annotation, and audio editing. **Online alignment**, also known as score following, performs the matching in real time while the audio is being received. Since it cannot rely on future information in the same way, online alignment typically has to trade off latency against precision.

Beyond this temporal distinction, alignments may be carried out at different levels of granularity. The three most common are **note-level**, **beat-level**, and **segment-level** alignment.

Note-level alignments are score matching processes that aim to synchronize at the individual note level. At its core, note-level alignment is based on note and onset/offset detection. In MIR literature it can also be referred to as “fine-grained alignment” because of the precision it requires. Contemporary audio-to-score alignment research focuses on this type of alignment

not only because of its precision but also because it can retrieve data relevant to other MIR fields and tasks. However, pure note-level alignments might falter if the performance deviates from the score, for example if the performer skips, repeats, or adds notes or score sections. Repeats and jumps are a well-established source of difficulty in score-performance synchronization [**fremereyHandlingRepeats2010**]. For this reason, some score-matching solutions combine fine-grained alignment with other methods.

Piano Precision is an example of an application of a link-based note-level score-alignment method. While the recording is playing Piano Precision shows a spectrogram and waveform that follow along a conventional playback cursor while, on a side window that contains the score, highlighting the corresponding elements being played in the score, note by note. This enables annotations to be mapped to both the audio and individual score elements simultaneously. The authors note the usefulness of this tool for performance researchers [**jiangPIANOPRECISIONADVANCING2025**] however, we will make the case that such feature is useful for BT and MIR research more broadly.

Beat-Level alignments aim to follow along beat and/or measures dictated in the score, and then linking them to the performance. Early score aligners [**duanSoundprismOnlineSystem2011**] would rely on such methods. Today, score alignment solutions, although mostly are note-level types, still combine Beat-Level alignment layers within their algorithms. A good example of this comes from [**peterAutomaticNoteLevelScoretoPerformance2023**], in it the authors improved on other alignment models, one strategy employed was adding analysis of "alignments at both the beat and the bar level". The beat level alignment would serve as "anchor points" in order to prevent "cascading errors, as misalignment will not propagate beyond the next anchor point", they noted that this improved results of automatic alignments.

Segment type alignments, map broad portions of performance audio to corresponding visual blocks in the score. This type of score matching is mostly made manually, it is common to see it in follow-along videos to help viewers listen to the music while visualising the score. This type of alignment is considered "weak but musically meaningful" [**jungUnifiedCrossmodalTranslation2026a**] since segments can be deliberately chosen i.e., to better show melodic or harmonic structure.

Among the visual alignment tools relevant to this thesis, ScoreWarp is especially important. ScoreWarp presents a score that is tied to a physical time axis, where each musical element is shifted so that its visual position coincides with the time at which it sounds [**goeblLetsScoreWarpAgain2025a**]. In other words, it implements a time-warped visualisation of the score while preserving the semantic structure of the original MEI data. This makes ScoreWarp a strong example of how alignment can be used not only as an algorithmic task, but also as a visual framework for interacting with music information. In the next chapter, we will return to this tool in more detail, since it became one of the key references for the development of this thesis.

2.4 Summary

This chapter provided a comprehensive overview of MIR visualisation techniques and tools, establishing the foundation for understanding audio-to-score alignment.

We began by categorizing music visualisations into three main types: traditional sheet music representations, symbolic machine-readable formats, and audio-based representations. This established the importance of MEI and Verovio, as well as the use of SVG in this context.

We then examined the contemporary landscape of MIR visualisation tools, distinguishing between GUI-based solutions such as Sonic Visualiser and Piano Precision, which provide user interfaces for audio analysis, and code-based workflows that afford researchers greater flexibility to develop specialized visualisations tailored to specific research questions. Python-based approaches using libraries such as Librosa and Matplotlib have become widely used in MIR research, particularly because of their integration with machine-learning pipelines and reproducibility requirements.

Finally, we introduced one of the core tasks of this thesis: audio-to-score alignment. This process synchronizes musical performances with their corresponding symbolic scores. ScoreWarp emerged as a particularly relevant tool, demonstrating how MEI and Verovio can be leveraged to create visually warped scores that maintain semantic integrity while aligning with performance time.

Chapter 3

Methodology

This chapter presents the methodological choices that guided the development of this thesis. It explains the overall research approach, the iterative process used to build and refine the work, the scope boundaries that define what the project does and does not cover, and the criteria used to evaluate the end result.

3.1 Research Approach

This thesis follows a design-oriented and implementation-driven research approach, supported by iterative prototyping. Rather than treating the problem as a purely theoretical exercise, the work focuses on building and refining a practical framework for time-aligned music visualisation that responds to the needs identified in the literature review and in the reference tools analysed in the previous chapter. The research process is therefore grounded in making, testing, and adjusting concrete visual and interaction decisions as the project evolves.

The methodology combines conceptual analysis with software development. First, the problem space was studied through the comparison of existing tools, visualisation strategies, and alignment workflows. That understanding was then used to define a focused system design that could support layered, time-aligned representations of musical data. From that point onward, implementation and evaluation proceeded together: each prototype was used to verify whether the proposed interaction model, visual structure, and alignment logic were adequate for the thesis goals.

This approach is appropriate because the main contribution of the thesis is not only to describe a concept, but to demonstrate how such a concept can be materialised into a workable framework. As a result, the methodology emphasizes practical feasibility, visual clarity, and alignment consistency, while still remaining informed by the academic context established in the literature review.

3.2 Iterative Development Process

The development of LayerIt followed an iterative path made up of five prototype cycles before the final framework took shape. Rather than moving from a fixed specification directly to implementation, the project evolved through successive experiments in which each prototype was built to answer a specific question, evaluated for what it revealed, and then either discarded or used as the basis for the next step. Seen in this way, the value of the iterations is not only in the software they produced, but in the design knowledge they accumulated.

We started by creating **BeatMapJS**. Its goal was to explore a web-based beat-annotation interface and to test whether existing beat-tracking tools could be combined with a more interactive annotation workflow. The prototype was built in JavaScript and HTML using BeatThis [**foscarinBeatThisAccurate2024a**] and Wavesurfer.js, and it included an interactive spectrogram, region annotation, a tempo-annotation table, and import/export support for SV-compatible text files. The main lesson was that, although the system was functional, it did not contribute a meaningful new annotation capability beyond the integration of BeatThis. More importantly, it did not produce a reusable Python library, which, from the start, was an important requirement. The decision carried forward was that a reusable Python core would be mandatory.

The second cycle was **MEI-Basic-Edit**. Its goal was to test whether manual score alignment could support audio-to-score annotation by allowing score elements to be adjusted against an audio layer. The prototype was built in JavaScript and HTML with Verovio, MEI, and SVG, and it rendered a score below a static spectrogram while allowing users to drag score elements horizontally to align them with the audio representation. It also included an embedded XML editor for direct manipulation of the underlying score code. The key lesson was that interactive score editing was expensive both conceptually and technically: it increased UI complexity and required a level of implementation effort that was not justified by the results. At the same time, this cycle provided valuable familiarity with MEI, Verovio, and SVG structure. The decision carried forward was that SVG would remain a viable composition substrate, but interactive editing would not be the main direction.

The third cycle was **BeatPrecision**, a fork of Piano Precision. Its goal was to extend an existing GUI environment with beat-annotation features while preserving the score-alignment functionality already present in the software. The prototype was built in Qt/C++, integrating BeatThis and implementing horizontal score-alignment alongside the existing spectrogram view. While functional, the project revealed significant challenges. Piano Precision's (and by extension Sonic Visualiser's) codebase proved complex and difficult to manage. Long compile times and the sheer size of the codebase made each developed modification increasingly challenging to implement, manage, and debug. More importantly, this iteration taught us a critical lesson: the need to distinguish between the interaction and visualisation problems in BA. We realized we could not tackle both simultaneously and would need to choose a direction. Recognizing a clear gap in the field, we opted to focus on the visualisation aspect, as we saw greater potential for the thesis to benefit the community in this area.

The fourth cycle was **Score-Alignment.py**. Its goal was to explore score-aligned spectrograms in a Python environment and to assess the Modusa framework as a possible basis for the project. The prototype was built in Python with Modusa and Matplotlib and produced an interactive spectrogram. The lesson was that a truly fluid interactive visualisation, with zooming, panning, a timeline, and playback control, was much harder to implement than expected. Even though the prototype worked, the result was not satisfactory enough to justify continuing in that direction, and the return on effort was not viable. The decision carried forward was to abandon real-time interactivity and instead prioritize static, high-quality, reproducible output.

The fifth cycle was **Score-Alignment.js**. Its goal was to revisit the aligned-spectrogram idea using the lessons learned from the earlier MEI and beat-annotation prototypes. The prototype was built in JavaScript and HTML with Verovio, MEI, and Wavesurfer.js, and it displayed a MEI score alongside a spectrogram while storing beat timestamps in the MEI data and highlighting notes on interaction. The lesson from this final exploratory cycle was that the idea was promising, but it also confirmed the need for a tool that was strictly focused on visualisation, written in Python, and organized using OO principles so that it could be reused and extended more easily. The decision carried forward was that this combination of requirements defined what would then be LayerIt.

Table 3.1: Summary of development iterations

Iteration	**Goal**	**Tools Used**	**Lesson**	**Conclusion**
BeatMapJS	Explore web-based beat annotation	JS/HTML, BeatThis, Wavesurfer.js, interactive spectrogram, region annotation, tempo table, SV-compatible import/-export	No new annotation feature beyond BeatThis; no reusable Python library	Reusable Python core is mandatory
MEI-Basic-Edit	Test manual score alignment against audio	JS/HTML, Verovio, MEI, SVG, drag-to-align score, XML editor	Interactive score editing is costly; gained MEI/Verovio/SVG familiarity	SVG as composition substrate; no interactive editing focus
BeatPrecision	Extend GUI beat annotation in an existing score-aligned tool	Qt/C++, BeatThis, horizontal score follow-along	Codebase was hard to extend; interaction and visualisation must be separated	Focus on visualisation, not the interaction problem
Score-Alignment.py	Explore score-aligned spectrograms in Python	Python, Modusa, matplotlib, interactive spectrogram	Fluid interactivity proved impractical	Prefer static, high-quality, reproducible output
Score-Alignment.js	Revisit aligned spectrograms with richer score interaction	JS/HTML, Verovio, MEI, Wavesurfer.js, beat timestamps in MEI, note highlighting	Promising, but confirmed need for a Python OO visualisation tool	Define the LayerIt conceptual baseline

Taken together, these five cycles show a deliberate narrowing of scope. The project moved from an interactive beat-annotation GUI toward a reusable OO visualisation framework. Each

prototype clarified one part of the problem and ruled out another, so the final system could be built on a more stable understanding of what the thesis needed to solve.

3.3 Scope Decision

The development process described above led to an important scope decision. At the outset, the project was framed broadly as a possible BA/BT graphical user interface, with both annotation and visualisation treated as potential contributions. However, the iterative prototypes made it clear that these were, in practice, two different problems, each with its own technical demands, design goals, and implementation costs. Attempting to solve both within a single master's thesis would have diluted the contribution and made it harder to reach a result that was both robust and meaningful.

For that reason, the project deliberately narrowed its focus from an interactive BA/BT GUI to an OO visualisation framework. The interaction problem proved too broad to address properly alongside the visualisation problem, while the visualisation side revealed a clearer and more relevant gap: the lack of a reusable, flexible, and well-structured Python tool for producing layered time-aligned musical data visualisations. By focusing on this gap, the thesis could concentrate on a contribution that was both feasible and valuable.

As a result, this thesis does not attempt to provide a complete BA/BT annotation environment, nor does it seek to solve the broader interaction problem in full. Instead, it focuses on the visualisation dimension, with particular emphasis on clarity, modularity, and alignment consistency. This narrower scope makes it possible to develop a more coherent contribution and to evaluate it against well-defined criteria.

3.4 Evaluation Criteria

The evaluation of this thesis is guided by a small set of criteria chosen to reflect the actual purpose of the framework. Since the goal is not to produce a general-purpose MIR platform, but rather a reusable visualisation tool, the evaluation focuses on whether the final system succeeds in covering the visualisation cases that motivated its development, while keeping the implementation manageable and the output reliable. In this sense, the framework is judged less as a standalone demo and more as a deliberate response to a specific research and design problem.

Firstly, the framework should be able to represent the kinds of score-aligned visualisations required by the thesis, including the layered combination of symbolic score data with audio-derived and annotation-derived information. Secondly, it should support modular composition and regeneration: existing visual layers should be combined, modified, and re-rendered through a repeatable procedure rather than manual image alignment. Thirdly, the output should be reproducible. Because the thesis is situated in a research context, the visual outputs should be generated through a process that can be repeated, inspected, and shared without ambiguity.

Two further criteria are extensibility and fidelity. Extensibility refers to whether the framework can be adapted to new visual layers or new combinations of existing ones without requiring major restructuring. Fidelity refers to whether the resulting figures preserve the musical and temporal relationships they are intended to represent, while still making clear the limits of the visual abstraction. In other words, the evaluation must consider not only what the framework can display, but also where its representation becomes less precise, less general, or less appropriate. Defining these criteria here makes the evaluation of the thesis read as a deliberate design assessment rather than as a mere presentation of the implemented tool.

Chapter 4

LayerIt
Design and Implementation

This chapter presents LayerIt as the concrete outcome of the research process described in the previous chapter. Its role is to show how the thesis moves from a problem statement and a set of methodological decisions to a working framework for composing score-aligned MIR visualisations. The chapter is organised from the problem definition to the system overview, then to the design choices that shaped the implementation, and finally to the mechanism used to combine the different visual layers into a single result.

4.1 The Problem and Our Solution

The central problem addressed by LayerIt is that score-aligned MIR figures are often created through a mixture of manual editing, *ad hoc* scripting, and tool-specific workarounds. In practice, this makes it difficult to produce figures that are reusable, reproducible, and easy to adapt when the underlying data changes. Existing GUI tools are useful for inspection and annotation, but they are not designed to support the kind of compositional, publication-oriented visual output required by this thesis.

LayerIt responds to this problem by treating visual elements as composable layers rather than as a single fixed image. The framework provides a structured way to generate, combine, and align music-related visual materials in SVG-based output. ScoreWarp provides the alignment backbone. LayerIt's contribution is an OO layer-composition model that supports the creation, modification, and regeneration of visual compositions.

4.2 LayerIt Overview

LayerIt is a Python framework designed to compose score-aligned MIR visualisations from independent visual components. At a high level, it takes aligned musical data, generates the corresponding visual representations, and arranges them into a layered figure that preserves the tem-

poral relationship between the score and the accompanying audio-derived or annotation-derived information.

The framework is modular: the same base structure can support different combinations of score material, spectrograms, annotations, beat information, and other derived data. These combinations can be modified and regenerated through scripts rather than reconstructed manually in an image editor.

The project uses librosa [**mcfeeLibrosaLibrosa01102025**], matplotlib [**thematplotlibdevelopmentteamMatp**] and Modusa [**modusa**] to generate audio and data visualisations. The score layer is rendered through ScoreWarp. The final output is an SVG file that combines the score with audio and data layers. The codebase also integrates BeatThis [**foscarinBeatThisAccurate2024a**], illustrating how precomputed BT output can be incorporated to produce beat-data visualisations.

4.3 Design Choices and Rationale

LayerIt uses matplotlib to generate audio and data representations when SVG is the final output format. Verovio, and subsequently ScoreWarp, render the score in SVG. Exporting matplotlib output to SVG therefore provides a practical route for combining chart-based visualisations with the rendered score.

SVG is a vector format, so representations that can be expressed as Cartesian plots, such as continuous curves and time-based annotations, can be exported without resolution loss. Other representations, particularly heatmaps such as spectrograms, are ordinarily rendered as raster images. Although SVG can embed such images, it is a static format and does not provide the interactive zooming and panning of a dedicated visualisation interface. LayerIt therefore supports PNG output for raster-based plots. This trade-off preserves the resolution independence of the score and vector plots, while raster layers may lose detail when enlarged and require greater storage.

Because SVG is a text-based markup format, it can also be inspected and modified directly. It can therefore serve as an intermediate representation between codebases written in different languages, such as Python, JavaScript, or C++. This property supports the potential integration of LayerIt into future MIR applications.

LayerIt was developed as a Python-based OO tool using established MIR libraries, including librosa and matplotlib. This choice keeps the framework compatible with common Python-based research workflows.

4.4 Generating Visuals

LayerIt was developed to generate visualisations for BA and BT tasks, so the current version focuses on these uses. It uses audio-to-score and data alignment to place relevant representations on a common visual timeline. In its current state, the codebase relies on the following files to generate complete visualisations:

File	Description
`.wav`	File with the audio recording of the performance
`.mei`	XML Score of the musical piece
`.maps.json`	Contains data that links onsets with score elements of the .mei
`.svg`	The score in visual format generated by ScoreWarp (requires .mei and .maps *a priori*)
`.npz`	File with algorithmic BT data

Table 4.1: Files required by LayerIt to generate complete visualisations.

4.4.1 Getting the Warped Score

The warped score can be generated using the ScoreWarp online demo. The user uploads the score in `.mei` format together with the MAPS file, then downloads the resulting score in `.svg` format.

The `.mei` file contains the score in XML format. A score in `.musicxml` format can be converted to `.mei`, for example with the Verovio online editor or MuseScore. In either case, the original score must be available in `.musicxml` format before conversion.

4.4.2 Getting the MAPS file

The MAPS file records performance onsets and links them to the corresponding score elements in the `.mei` file through their `xml:id` attributes. Two methods can be used to obtain this file.

The first method, described in [**goeblLetsScoreWarpAgain2025a**], uses the MIDI-to-MIDI matching algorithm of [**nakamuraPERFORMANCEERRORDETECTION**]. It requires both the `.mei` score and a MIDI performance. Examples from the ASAP dataset [**peterAutomaticNoteLevelScoretoPe** can facilitate this process. When a MIDI performance is unavailable, the MAPS file must be generated independently.

The MAPS file is a `.json` file. For the current version of LayerIt, it must provide score and performance onset data in the following form:

Listing 4.1: Minimal structure of the MAPS.json file to work with LayerIt and ScoreWarp

```
{
    "obs_mean_onset": 0.853333333,
    "xml_id": ["x1iw1eq1"],
    "obs_num": 1
}
```

In this example:

- `obs_mean_onset` is the time in seconds of a given onset.
- `xml_id` is the `xml:id` of the corresponding onset note in the `.mei` score. For a chord onset, this attribute can contain more than one `xml:id`.

- `obs_num` is a sequential counter for the supplied onsets.

LayerIt relies on MAPS in two complementary ways: directly for onset visualisation, and indirectly through ScoreWarp, which requires MAPS to generate the warped score aligned with the timeline SVG element which constitutes the alignment reference for all overlaid data.

MAPS is a purpose-specific alignment representation for this workflow. More general interchange formats, such as the match file format, encode note-level correspondences between scores and performances [**foscarinMatchFileFormat2022**]. LayerIt does not currently read or produce match files; supporting such formats would broaden the range of alignment pipelines that can provide its input data.

ScoreWarp performs best when MAPS contains dense onset annotations across the full performance. However, it can still align scores when annotations are sparse via interpolation between available onsets and their @xml.id anchor points. In practice, this allows different precision levels (i.e., downbeats, phrase boundaries, or measure-level anchors), trading annotation effort for alignment precision. Even minimal anchors (first and last onset) may be usable for short performances or segments, but intermediate alignment becomes less reliable due to interpolation error.

For the examples provided with LayerIt, PP was modified to obtain this data. PP's Piano Aligner plugin [**jiangPIANOPRECISIONADVANCING2025**] matches performance onsets to score elements defined by the `.mei` file. PP displays these correspondences in a Sonic Visualiser time-instance layer. The modified layer displays the `xml:id` of each onset rather than its score position, expressed as `measure + position_in_measure`. The time instances can then be exported as `.txt` files containing `obs_mean_onset` and `xml_id`, and converted into MAPS format using the `TXT_to_Maps()` function in `src/functions/warp_score`.

4.4.3 Providing BT data

Contemporary BT algorithms commonly produce a beat activation function: a continuous curve representing the estimated likelihood of a beat at each frame. Visualising this curve alongside the discrete beat estimates exposes the model output from which those estimates are derived and provides qualitative context for interpreting them.

In `src/functions/Beat_Layers`, the `Run_BeatThis()` function generates an `.npz` file containing the data required for visualisation. Although this function uses BeatThis, the `.npz` format can also act as an intermediate format for BT visual layers when it follows the structure used by `Run_BeatThis()`. The file should contain five key components:

Field	Purpose
`beat_times`	Time coordinates (seconds) aligned with probability frames
`beat_probs`	Continuous beat activation function values
`downbeat_probs`	Continuous downbeat activation function values
`detected_beats`	Discrete beat positions derived from peak detection
`detected_downbeats`	Discrete downbeat positions from peak detection

The activation fields capture continuous model outputs across time, while the detected positions represent the final beat estimates. This separation enables the visualisation of both the continuous activations and the discrete estimates.

4.5 Layer Alignment Method

LayerIt currently provides tools for generating visualisations for BT and BA tasks, but the framework is not limited to these applications. Layers can be SVG or PNG elements that span the same, or a proportional, width as a given timeline. Other SVG or PNG plots can therefore be incorporated when they use the same temporal extent.

When the warped score is generated, ScoreWarp can add a visual timeline for the performance. This timeline may span a greater width than the recorded performance and can extend beyond the visible SVG canvas. Because it provides the reference used to position score elements, LayerIt also uses it to align the visual layers with the recording.

LayerIt aligns layers as follows:

1. It reads the SVG containing the warped score, rendered by ScoreWarp, and retrieves the full length of the generated timeline in pixels.
2. It obtains the duration of the audio recording on that timeline.
3. It calculates the recording length in pixels by proportional scaling from the recording duration and the full timeline length.

In the codebase this is done through a single method:

- `Visualizer().get_Layers_WidthHeight()`

LayerIt does not yet provide conversion methods for arbitrary external visualisations. However, the alignment procedure can be applied to an external image by placing its left edge at the start of the relevant timeline and scaling it to the recording length calculated by the framework. The image can then be scaled to the timeline length of the score generated through ScoreWarp.

4.6 Layers on Layers, within Layers

LayerIt combines aligned visualisations of algorithmic data, audio, and symbolic music representations, such as scores, that share a common timeline. Its Python OO architecture represents these visualisations as layers that can be composed into larger outputs. This organisation supports the modification and regeneration of the visual compositions described in Chapter 5.

The visualisations generated by LayerIt follow a “layers within layers” philosophy: visual elements are self-contained components that can be composed into upper-level elements, which in turn can be composed into yet higher levels. Figure 4.1 demonstrates this methodology.

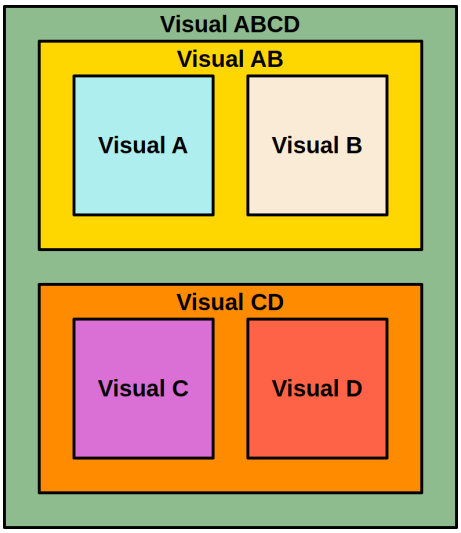

Figure 4.1: Composition of the LayerIt visualisation system. Each coloured box represents a distinct visual element that composes a higher-level visualisation.

This "layers within layers" design is applied at three levels of the project: the **visual level**, where visualisations combine aligned data, audio, and musical elements; the **SVG level**, where LayerIt organizes the XML structure of the output; and the **code level**, where the OO architecture organizes the corresponding program components. Together, these levels describe how individual elements are combined into composite visualisations.

4.6.1 Visual Level

At the visual level, layers are output elements aligned to a shared timeline. They can be arranged vertically in a composite view while retaining the temporal correspondence between the score, audio-derived representations, annotations, and BT output.

4.6.2 SVG Level

At the SVG level, LayerIt organizes the XML structure of the output so that individual and combined layers remain identifiable. The following listing shows a simplified representation of the XML structure produced for a composed SVG file:

Listing 4.2: Example structure of an SVG file

```
<svg>
|   <Top Visualisation>
|   |   <Layers>
|   |   |   <image spectrogram.png>
|   |   |   <BeatThis Output>
|   |   |   <Onsets>
|   |   <Score>
```

```
|   |   <timeAxis>
|   <Bottom Visualisation>
|   |   <Layers>
|   |   |   <image spectrogram.png>
|   |   |   <Beat Probability>
|   |   |   <Downbeat Probability>
|   |   <Score>
|   |   <timeAxis>
</svg>
```

This XML organisation allows the SVG to be inspected and modified directly through its markup. It also preserves an identifiable structure for code-based processing while keeping the SVG readable. The nesting of elements at the XML level mirrors the visual composition described above.

4.6.3 Code Level

At the code level, LayerIt's OO architecture organizes each visualisation component as a self-contained module with a consistent interface. The same composition principle used at the visual and SVG levels is therefore represented in the code structure. The following snippets illustrate the base abstraction, the `Visualizer` composition API, and a short end-to-end workflow.

Listing 4.3 shows an illustrative `Layer` base-class interface. The omitted imports are not relevant to the interface.

Listing 4.3: Layer interface (concise)

```python
class Layer(ABC):
  def __init__(self, name: str = "Layer"):
    self._data = None
    self.name = name

  @abstractmethod
  def load_data(self, **kwargs) -> bool:
    ...

  @abstractmethod
  def draw(self, ax, shared_data) -> tuple:
    ...
```

In LayerIt, a `Visualizer` instance composes multiple lower-level layers, which may represent data plots or audio representations, and provides them with a shared data dictionary through `load_all_layers()`. The next listing (4.4) illustrates this composition.

Listing 4.4: Visualizer Instance Composition

```
# Create a visualizer and compose layers
visualizer = Visualizer()

# Add multiple composable layers
visualizer.add_layer(BeatProbabilityLayer(color=(1, 0, 0)))
visualizer.add_layer(DownbeatProbabilityLayer(color=(0, 0, 1)))
# ...

# Load all data and render the combined layers
visualizer.load_all_layers(audio_path="performance.wav",
                           beat_file="beats.npz")
fig, ax = visualizer.draw()
```

Each `Visualizer` instance can be rendered to SVG or PNG. At this stage, the outputs are Matplotlib figures and can be displayed directly with `show()`. To combine them with other `Visualizer` instances and a score, they are exported to SVG. The intermediate SVG layer allows the aligned score to be added to each visualisation, producing a higher-level composite that can then be stacked with other rendered SVGs to create the final aligned view. The following illustrative listing shows this process.

Listing 4.5: Simplified workings for SVG layers.

```
# Export visualizers to SVG and align them to the warped score
SVG_Layer1 = Layer1.turn_to_SVG("Output/Path/And/Filename", svg_warped_score)
SVG_Layer2 = Layer2.turn_to_SVG("Output/Path/And/Filename", svg_warped_score)

# Add the score to an SVG layer
SVGLayer1_withScore = Layer1.combine_layers_with_score(filename, svg_warped_score)

# Stack SVG layers at defined vertical positions
EndVisualization = Visualizer()
svg_layers = [(SVG_Layer1_withScore, 50),(SVG_Layer1, 200), (SVG_Layer2, 350) ...]
EndVisualization.create_final_SVG(width, height, (...), svg_layers)
```

The example shows that higher-level SVG outputs can be positioned or stacked vertically, while lower-level Matplotlib layers can be composed into them. This compositional behaviour implements the “layers within layers” design of LayerIt.

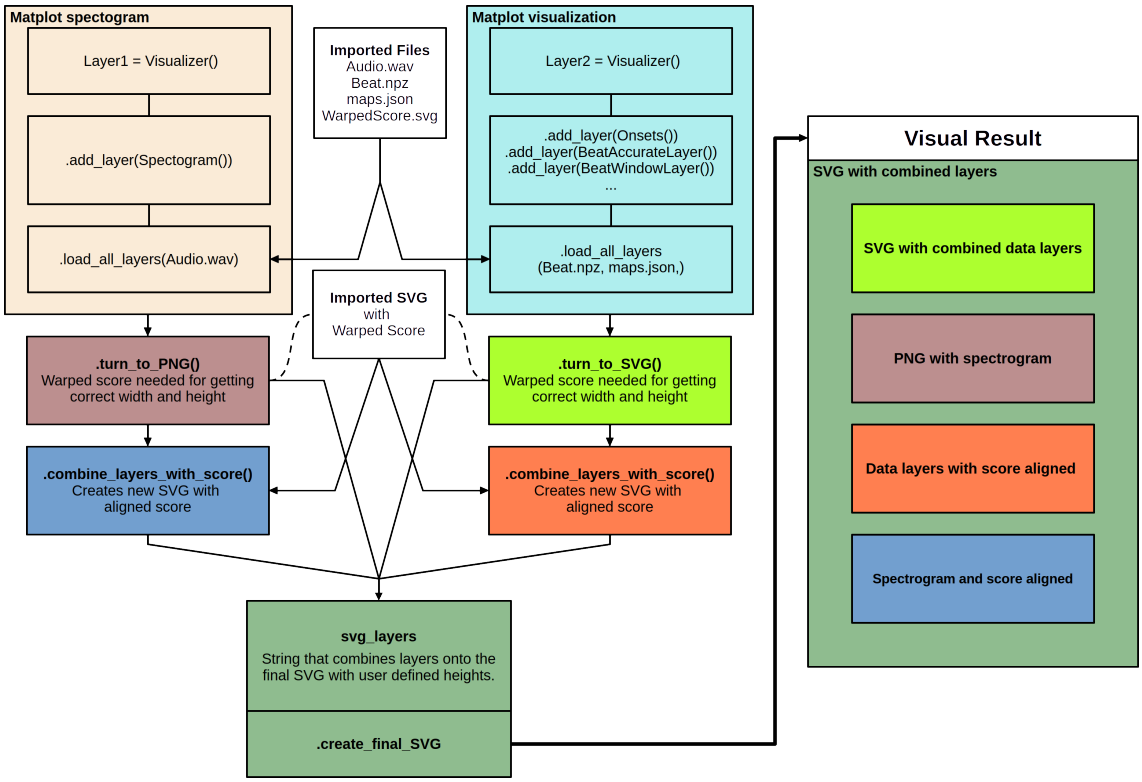

Figure 4.2: Complete LayerIt workflow: from Visualizer() composition to final stacked SVG visualisation. The diagram shows how multiple visualisation layers are created independently, exported to SVG format, combined with the warped score, and finally stacked vertically with defined y-offsets to produce the final custom composite visualisation.

LayerIt supports a workflow from matplotlib-based visualisation generation to SVG or PNG composition. The final composition stage can include externally produced graphics, allowing code-generated visualisations to be combined with images created outside the codebase.

This hybrid design supports multiple workflows, including Python-based prototyping and the use of externally generated images. Custom layer implementations and external image inputs can both be incorporated into the same composite visualisation.

These code-level choices provide the compositional mechanism evaluated in Chapter 5. The case study demonstrates that existing layers can be modified, exchanged, and regenerated through the common interface. The addition of entirely new layer types remains a direction for future work rather than a quantified result of this thesis.

4.7 Summary

This chapter closes the methodological and implementation stages of the thesis by connecting the development process described in Chapter 3 with the framework described in this chapter. The iterative path taken through the five prototypes clarified the scope of the work, showing that

the most meaningful contribution was not a new annotation system, but a reusable visualisation framework grounded in time-aligned music data.

LayerIt was then shaped to answer that need: a Python OO framework for composing score-aligned MIR visualisations through layered SVG-based output. Its design emphasizes reuse, composability, and visual consistency, while relying on ScoreWarp for the alignment backbone and on established MIR libraries for the generation of the underlying audio and data layers. Taken together, the two chapters show how the project moved from exploratory prototyping to a focused implementation that can now be assessed against the criteria established in the methodology chapter.

Chapter 5

Validation

This chapter evaluates LayerIt against the criteria defined in chapter 3: coverage, reproducibility, composability, and fidelity and limits. Since a controlled user study was outside the scope of this work, the evaluation is qualitative. It combines a capability demonstration based on figures generated with LayerIt, a contextual comparison with existing workflows, and an explicit discussion of the evidence that remains to be collected.

5.1 Validation Approach

The validation considers whether the framework can express the score-aligned, multi-modal figures that motivated the thesis, and whether its composition model supports their creation, modification, and regeneration once a score-performance alignment is available. The objective is not to claim that LayerIt replaces interactive annotation software, reduces annotation effort, or establishes the accuracy of an alignment algorithm. Instead, it assesses the framework as a reproducible means of composing visual material from modular layers.

All figures in this chapter are original outputs generated locally from the same case study: J.S. Bach the fuge from BWV 856. The inputs are the audio recording, its MEI score, a MAPS file, the ScoreWarp-generated SVG score, and BeatThis output. The figures are therefore not reproductions of published images. Published work is referenced only to motivate the figure types and the workflow gap addressed by the framework.

The evaluation distinguishes one-time preparation from per-figure work. Producing a score-aligned figure requires preparing the MEI score, MAPS annotations, warped score, and, where relevant, algorithmic output. These are substantial prerequisites shared by all LayerIt figures. Once they are available, the scripts supplied with this thesis define and compose the desired layers, and can be re-run when the input data or visual parameters change.

5.2 Reproducing Composite Figure Types (RQ1)

RQ1 asks how heterogeneous MIR visualisations can be combined in a common score-aligned representation. The four examples in this section address this question by composing symbolic notation, audio-derived representations, onset annotations, and BT output around the same warped score timeline. They cover both dense, multi-layer views and a time-based display.

5.2.1 Score, audio, and chroma on a shared axis

The first example combines a waveform, a time-warped score, a spectrogram, and a chromagram. Green dotted onset markers are repeated across the layers, making their common time reference visible. The figure demonstrates that the layer model is not limited to beat tracking: it can combine a symbolic score with multiple complementary audio representations in one vertically aligned view. This is the type of multimodal display used to relate score information to time-frequency and pitch-class information in MIR workflows [**mullerFundamentalsMusicProcessing2015**].

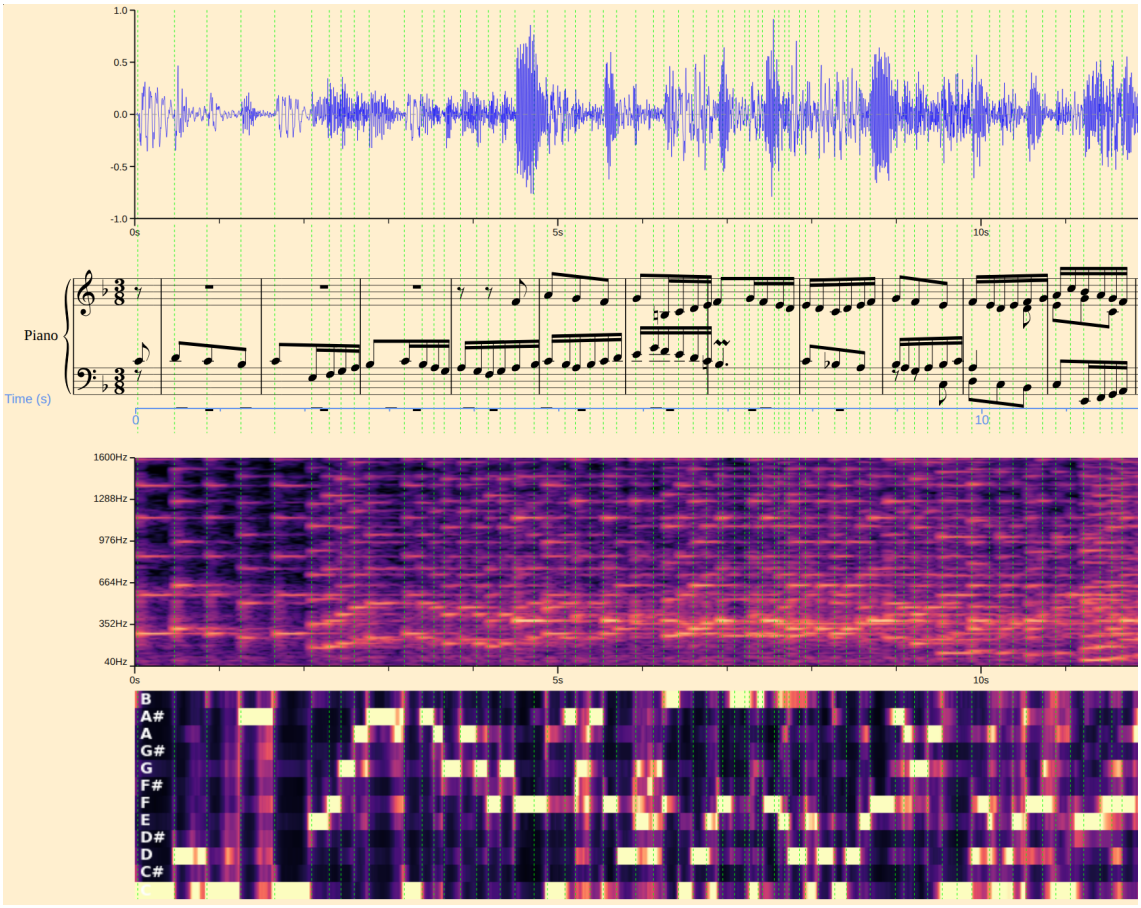

Figure 5.1: LayerIt composition of waveform, time-warped score, spectrogram, and chromagram for BWV 856. The green dotted lines show shared onset positions across all layers.

5.2.2 Score-aligned BT analysis

The second example places a spectrogram, BeatThis beat and downbeat estimates, onset markers, and the time-warped score in a single composition. The score provides musical context directly beneath the time-based data. This is valuable when interpreting a BT result because the reader can relate an estimated beat to the notated event, this provides a more accurate evaluation on the BT performance.

The figure type is motivated by the visualisation in Sá Pinto's BT evaluation work [**pintoShiftIfYoua**], where musical context and time-based analysis are complementary but not physically aligned. LayerIt does not change the evaluation procedure proposed in that work; it provides a way to present the relevant score, spectral, and beat information on a shared axis.

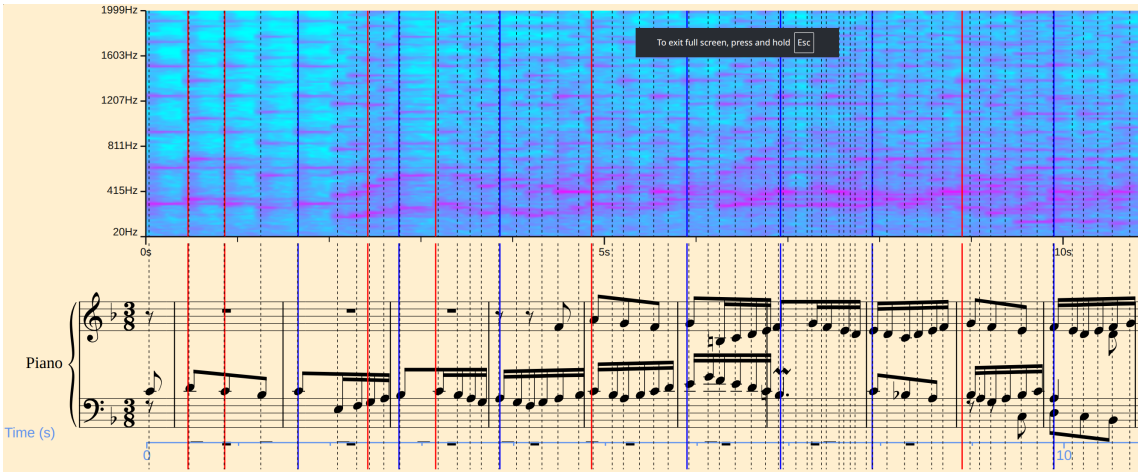

Figure 5.2: LayerIt score-aligned BT display for BWV 856. The spectrogram, beat (red) and downbeat (blue) estimates, onset markers (black dotted lines), and warped score share the same performance timeline.

5.2.3 Probability and position displays

BT analysis benefits from retaining both the discrete output of an algorithm and its frame-wise activation functions. Figure 5.3 presents two such views. In the upper view, detected beat and downbeat positions are displayed together with the corresponding probability curves. In the lower view, faded vertical windows indicate intervals in which the activation exceeds a configurable threshold. For this excerpt, the beat threshold is set to 20% and the downbeat threshold to 50%. In both cases, the curves, positions or threshold windows, and score remain tied to the same horizontal axis.

The thresholds were selected through iterative visual inspection to make the beat subdivision visible in this excerpt. The resulting windows agree with the expected subdivision shown by the score, illustrating how a researcher can tune the visualisation to inspect the behaviour of a BT system. More generally, varying the thresholds makes it possible to examine activation regions, compare them with selected positions and notated events, and diagnose whether a change in the visualised output reflects a meaningful pattern or a weak activation. The visualisation supports

qualitative analysis; the selected thresholds are presentation parameters for this case study and do not, by themselves, establish ground-truth correctness.

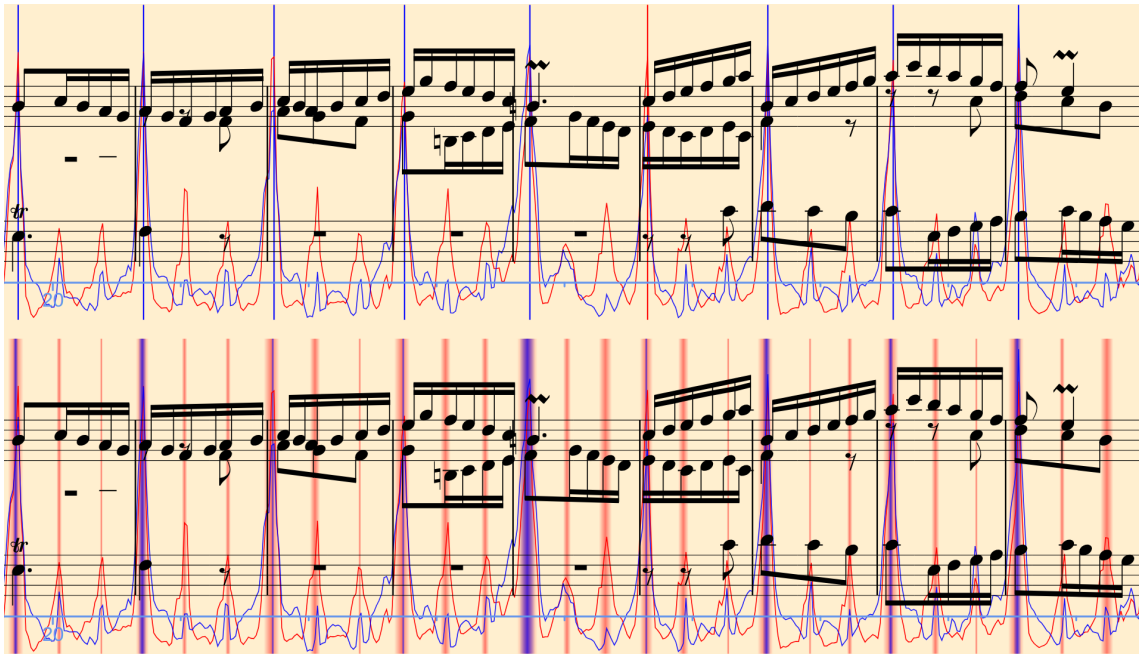

Figure 5.3: LayerIt probability-and-position displays for BWV 856. The upper composition shows BeatThis beat and downbeat positions with their activation curves; the lower composition replaces the discrete positions with faded activation-threshold windows, set to 20% for beats and 50% for downbeats in this excerpt.

5.2.4 Layered time-based display

The final example demonstrates the lightweight end of the composition model. It overlays a normalized waveform with BeatThis beat and downbeat positions, then places the warped score below it. Although it contains fewer layers than the previous examples, it uses the same composition process and preserves the same time relationship. This supports rapid inspection of how detected beats relate to both acoustic events and notated musical material.

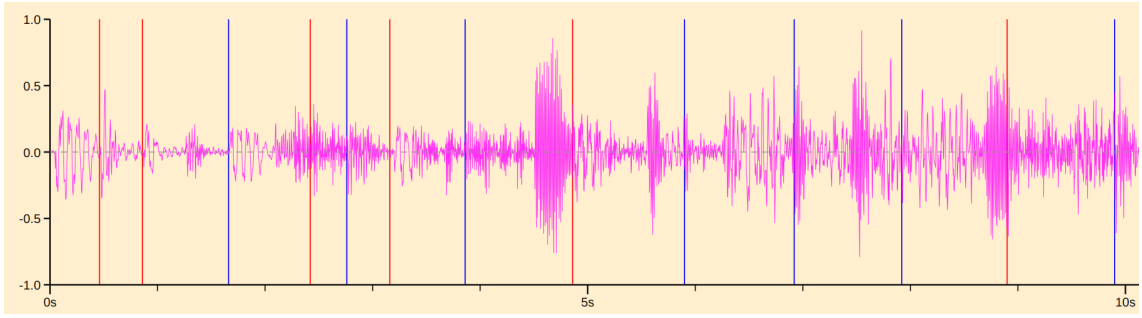

Figure 5.4: LayerIt waveform display for BWV 856, with BeatThis beat and downbeat positions aligned to the time-warped score.

Table 5.1: Composite figure types reproduced with LayerIt.

Figure type	**Motivating context**	**LayerIt layers**	**Composition result**
Score, audio, and chroma	Multimodal music representations [**mullerFundamentalsMusicProcessing2015**]	Waveform, onset markers, warped score, spectrogram, chromagram	Multiple audio representations aligned to the score
Score-aligned BT analysis	BT evaluation [**pintoShiftIfYoua**]	Spectrogram, beat and downbeat positions, onset markers, warped score	Musical context and algorithmic output in one view
Probability and position display	Qualitative inspection of BT output	Beat and downbeat probabilities, positions or threshold windows, warped score	Continuous confidence and discrete output compared on one axis
Layered time-based display	Compact BT inspection	Waveform, beat and downbeat positions, warped score	A minimal score-aligned analytical display

Taken together, these examples answer RQ1 at the level of framework capability: the layer model can compose heterogeneous visualisations into a common score-aligned representation. The evidence is a set of independently re-runnable scripts rather than a manually assembled figure. The examples do not establish universal visual design rules; they show that the required figure types are expressible by the framework.

5.3 Workflow Capabilities for Annotation and Evaluation (RQ2, RQ3)

This section positions LayerIt alongside Sonic Visualiser, Piano Precision, and manual SVG editing in relation to the workflows that motivated the project. It concerns the production and inspection of score-aligned figures, rather than a complete feature comparison of the other tools. Sonic Visualiser and Piano Precision remain purpose-built interactive applications, whereas LayerIt is a scriptable framework for generating static, structured SVG images.

5.3.1 Evaluation workflow

For BT evaluation, the principal advantage of LayerIt is that the algorithm's discrete positions and continuous activation functions can be read against the same time-warped score. Figure 5.3 illustrates this combination: the reader can inspect peaks in the beat and downbeat activations, the positions selected by the algorithm, and the corresponding notation without translating between separate time references.

This supports qualitative diagnosis. For example, a selected beat can be compared with a nearby activation peak and with the score event at the same horizontal position. Conversely, an activation peak that does not result in a selected position can be identified and interpreted in musical

context. Sonic Visualiser supports rich audio inspection, but the workflow considered here requires an additional score-aligned composition. Piano Precision provides score-performance links for navigation and analysis [**jiangPIANOPRECISIONADVANCING2025**]; however, its link-based score view does not provide the continuously time-warped score layer required by these static composite figures.

5.3.2 Annotation workflow

LayerIt is not an interactive annotation environment and does not itself let an annotator place or correct beat times. Its contribution to annotation is instead a visual reference for an upstream annotation workflow. An annotator can inspect the alignment between the spectrogram or waveform, onset markers, candidate beat positions, and the time-warped score; corrections are then made in the relevant annotation or MAPS source and the figure is regenerated.

In this workflow, the score is physically positioned on the same axis as the data to be inspected. This differs from a link-based score display, where correspondence is expressed through navigation or connections between separately laid out views. For tasks that require deciding whether a candidate beat coincides with a notated event, the shared-axis view provides direct visual context. It should nevertheless be understood as a diagnostic and communication aid, not as evidence that LayerIt improves annotation speed or accuracy; such a claim would require an empirical user study.

Table 5.2: Workflow characteristics relevant to the score-aligned visualisation task considered in this thesis.

Dimension	**Sonic Visualiser**	**Piano Precision**	**Manual SVG editing**	**LayerIt**
Shared score time axis in the workflow considered	Not provided	Link-based score view	Possible through manual placement	Provided through ScoreWarp
Arbitrary algorithmic curves	Limited by available layers/plug-ins	Outside the workflow considered	Possible through manual construction	Provided through Layer subclasses
Scriptable figure generation	Not assessed	Not demonstrated in this study	Depends on a separately maintained procedure	Provided through the composition script
Regeneration after data change	Manual reconfiguration	Manual view recreation	Repeat manual editing	Re-run with updated inputs
Adding a visualisation type	Plugin development	Plugin development	Manual construction	Implement or configure a Layer

No controlled count of manual operations or wall-clock time was recorded for the workflows considered here. Consequently, this thesis does not report numerical claims about time saved by LayerIt. The evidence for RQ2 is functional rather than comparative: once the inputs are prepared, the supplied scripts generate the demonstrated figures without manual image alignment, and the

same scripts can be re-run after changes to the data or visual parameters. A future study should measure one-time setup and per-figure effort separately.

5.4 Modular Composition Evidence (RQ2)

The architecture described in chapter 4 defines a common `Layer` interface and composes layers through `Visualizer`. The examples in this chapter exercise that interface with several independent layer types including: spectrogram, chromagram, waveform, onsets layer, BT output layers, beat/downbeat activation curves. Each is added through the same `.add_layer()` operation and loaded through the common `.create_final_SVG()` workflow.

The scripts provide practical evidence of composability: Figure 5.1 adds waveform, chromagram, and onset layers to the same output, while Figure 5.3 substitutes discrete position layers with probability-window layers without changing the composition mechanism. This is consistent with the modular claim of Chapter 4.

The current evidence supports the claim that existing layer types can be composed and exchanged through a common interface. It does not quantify the implementation cost of adding a completely new visualisation type, because no new layer was introduced solely for this evaluation with a measured line count and a documented change to the framework core. Such a probe should be added in future work by implementing one new layer and documenting whether any change outside that layer is required.

5.5 Limitations and Difficult Cases (RQ3)

The examples in this chapter depend on a reasonably faithful correspondence between the performance and the score. LayerIt composes and scales visual layers against the ScoreWarp timeline, so the fidelity of the composite visualisation is bounded by the quality and density of the underlying alignment data. This is an important distinction: a misleading score position caused by an inaccurate MAPS alignment is not a failure of the layer-composition mechanism, but the final figure nevertheless inherits that limitation.

5.5.1 Expressive performances

Highly expressive performances, including substantial rubato, skipped material, repetitions, or departures from the notated score, can make the relationship between score position and performance time difficult to represent with a simple warped timeline. Structural discontinuities such as repeats and jumps are a recognised challenge for score-performance synchronization [**thomasLinkingSheetMusic2012**, **fremereyHandlingRepeats2010**]. In such cases, a score element may be placed at a time that does not adequately describe the local musical event, and the misalignment then propagates visually to every layer that uses the same reference.

No dedicated expressive-performance stress test was completed for this thesis. The present evidence therefore establishes this as a boundary of the approach rather than measuring a specific

degradation pattern. A suitable follow-up evaluation would compare dense and manually verified MAPS annotations for an expressive performance with a less expressive reference performance, then report where the warped score remains informative and where it ceases to be reliable.

5.5.2 Sparse onset annotation

Sparse MAPS annotations present a related limitation. ScoreWarp can interpolate between available anchors, reducing the human effort required to prepare a score-performance alignment. However, the temporal position between widely separated anchors is then an interpolation rather than a directly supported observation. The resulting score placement may be sufficient for an overview, but it is less appropriate for detailed beat-level inspection in passages with tempo variation.

The current case study uses the available MAPS data and does not include a controlled dense-versus-sparse comparison. Thus, the trade-off is identified from the operating assumptions of the pipeline, as described in chapter 4, rather than quantified here. A future evaluation should generate dense and sparse MAPS versions of the same excerpt and compare the resulting score positions directly.

5.5.3 Manual editing of the final SVG

Manual editing of a final SVG is not required for visual customisation in LayerIt. The framework exposes customisation through its layer configuration and composition code, allowing users to adjust existing layers and incorporate the visual elements required for a particular analysis while retaining a re-runnable generation script. It can also include externally prepared PNG and SVG assets, allowing a composite visualisation to combine LayerIt-generated layers with other image-based material.

When an existing layer does not provide the required behaviour, the OO architecture supports modifying or extending the codebase by implementing a new `Layer` or adapting an existing one. This keeps the visual logic within the framework and preserves traceability from the source data and configuration to the resulting figure. Direct hand-editing of the exported SVG remains possible as a fallback, but it should be avoided when the adjustment can instead be represented in code, since unrecorded manual changes weaken reproducibility. If such editing is necessary, the modified SVG and the reason for the modification should be retained alongside the generation script.

5.6 Summary

This chapter provides a qualitative answer to the three research questions. For RQ1, the four figures demonstrate that LayerIt can compose symbolic score material, audio representations, onset annotations, and BT output into common score-aligned views. For RQ2, the generated examples and composition workflow show that existing layers can be created, modified, exchanged, and regenerated through a common scriptable interface.

For RQ3, the examples show how a shared-axis visualisation can support qualitative inspection of BT results and can serve as a diagnostic reference for annotation. LayerIt is not itself an interactive annotation tool, and no user-study claim is made. Its usefulness is also conditioned by the quality of the score-performance alignment: expressive performances and sparse annotation anchors remain important limitations that require dedicated empirical evaluation.

Chapter 6

Conclusions and Future Work

6.1 Achievements

This thesis set out to address the lack of a reusable framework for composing score-aligned, multi-modal MIR visualisations and to demonstrate how such a framework can provide qualitative context for beat annotation and BT evaluation workflows. Revisited against the objectives and research questions set out in chapter 1, the work can be summarised as follows.

- **Objective 1** was met through the development of LayerIt, a Python OO framework in which score, audio, and algorithmic data are treated as independent, composable layers. chapter 4 showed that this "layers within layers" philosophy holds consistently at the visual, SVG, and code levels, giving the framework a coherent architecture rather than a single-purpose script.
- **Objective 2** was met through the generation and regeneration of four distinct composite figure types, ranging from dense multi-layer views combining a warped score with waveform, spectrogram, and chromagram to lightweight waveform-and-beat displays. The same composition mechanism creates the figures, and the demonstrated layers can be modified or exchanged through the common Layer and Visualizer interfaces. This provides functional evidence for RQ2 without making comparative claims about development time or user effort.
- **Objective 3** was met through a case study built around BT output for BWV 856. RQ3 was addressed by demonstrating how a shared-axis visualisation provides qualitative context for interpreting beat-tracking results and inspecting beat annotation data. This context depends on the score-performance alignment: highly expressive performances and sparse onset annotations remain important limitations, so LayerIt is not presented as an interactive annotation tool or a universal solution.

Beyond the framework itself, this thesis produced concrete, reusable outputs: LayerIt as an open-source, pip-installable Python library; an open-source repository of the prototypes developed throughout the project; and a Late-Breaking Demo paper submitted to ISMIR 2026. Just as

importantly, the iterative process described in chapter 3 is itself an achievement of the work: by building and discarding five successive prototypes, the thesis identified that beat annotation and BT visualisation are, in practice, two separate problems, and it deliberately narrowed its scope to the one that offered the clearest and most feasible contribution. That decision allowed LayerIt to be evaluated against criteria appropriate to a visualisation framework rather than an interactive GUI.

6.2 Future Work

The limitations identified in chapter 5 point directly to the most pressing follow-up work. First, the current evidence demonstrates the composition of existing layers but does not quantify the effort required to introduce a completely new layer; a controlled extensibility study could implement a new `Layer` subclass and document the changes it requires. Second, the fidelity of LayerIt's visualisations is bounded by the quality of the underlying score-performance alignment; dedicated experiments comparing dense and sparse MAPS annotations, and stress-testing the framework against expressive performances with substantial rubato or structural deviations from the score, would clarify precisely where the time-warped representation remains reliable.

Beyond validating what already exists, several directions could extend LayerIt's scope. The framework currently integrates a single alignment method (ScoreWarp) and a single BT algorithm (BeatThis). A synchronisation toolkit such as Sync Toolbox could provide additional upstream score-performance alignments [**mullerSyncToolbox2021**], while support for an interchange representation such as the match file format could make these inputs more portable [**foscarinMatchFileFormat2022**]. Supporting these and other alignment and MIR algorithms as further `Layer` implementations would test the framework under more demanding conditions and grow LayerIt into a more general-purpose visualisation toolkit. The case study in this thesis was also limited to a single solo piano excerpt; validating the framework against a broader range of instrumentations, genres, and performance styles would strengthen the generality of its conclusions. Finally, having deliberately set aside the interaction problem in chapter 3, a natural long-term direction is to revisit it: the SVG output produced by LayerIt already preserves the addressable structure inherited from Verovio and MEI, which could be leveraged to build an interactive, browser-based annotation layer on top of the visualisations this thesis already knows how to generate. Pursuing this direction would reconnect the visualisation contribution made here with the beat-annotation and BT-evaluation tools that originally motivated this thesis.

6.3 Final Remarks

This thesis began with the observation that producing time-aligned, multi-modal visualisations can require the manual combination of separately generated score, audio, and algorithmic representations. Through an iterative development process and careful architectural design, we developed

LayerIt, a Python-based OO tool for composing these representations as structured, score-aligned visualisations.

LayerIt's value lies in its demonstrated ability to compose visual layers into re-runnable, score-aligned figures for BT evaluation and beat annotation inspection. By grounding the system in established conventions, including OO design patterns, common MIR libraries, and standardised formats, the framework provides a basis for future extension and integration.

As MIR research continues to evolve through advances in machine learning, increasing dataset complexity, and growing interdisciplinary collaboration, the need for flexible visualisation tools will continue to grow. LayerIt provides a focused foundation for future work on composable score-aligned visualisations.